Aubrée Romain **Beauvallet**
07/11/2026 **Semestre 3**
SLAM

Groupe 1

Bloc 2 TD 2 - Les Triggers

Contexte **2**
Exercice 1 : parc immobilier **2**
Exercice 2 : des produits au prix variant au fil du temps **8**
Exercice 3 : location de véhicules **15**
Conclusion **19**

Contexte

Ce TP porte sur les triggers (ou déclencheurs). Un trigger est un bloc de code que le SGBD exécute tout seul avant ou après une insertion, une modification ou une suppression sur une table. Il sert à faire respecter des règles de gestion que les simples contraintes de clés ne couvrent pas.

On travaille sous MySQL / MariaDB, administré avec phpMyAdmin. Dans un trigger, le mot-clé NEW désigne la ligne en cours d'insertion ou de modification, et OLD la ligne avant modification ou suppression. Pour interdire une opération, on lève une erreur avec SIGNAL SQLSTATE '45000'.

On traite les trois exercices du sujet : un parc immobilier, une boutique de produits et de formations, puis un parc de location de véhicules.

Exercice 1 : parc immobilier

On part du modèle fourni : un Immeuble contient des Appartement, chaque appartement est composé de Piece et illustré par des Photo. Les quatre tables sont créées sous MySQL avec le script du sujet, puis on ajoute les triggers demandés.

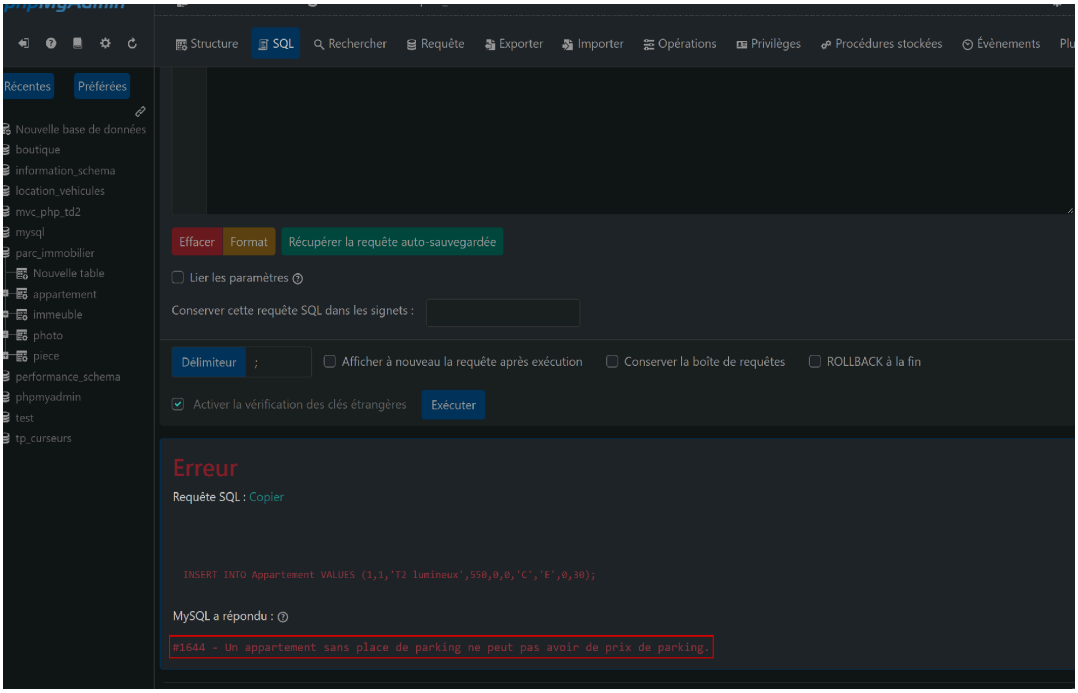
Question 1 : prix du parking

La règle : le prix de la place de parking doit être NULL quand l'appartement n'a pas de place, et renseigné quand il en a une. On écrit un trigger BEFORE INSERT sur Appartement qui vérifie cette cohérence.

```
CREATE TRIGGER trg_appartement_avant_insert BEFORE INSERT ON Appartement
FOR EACH ROW
BEGIN
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Un appartement sans place de parking ne peut pas avoir de
prix de parking.';
    END IF;
    IF NEW.placeParking = 1 AND NEW.prixParking IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Un appartement avec place de parking doit avoir un prix
de parking.';
```

```
    END IF;
END
```

Test : on tente d'insérer un appartement sans place de parking mais avec un prix. Le trigger bloque l'insertion.



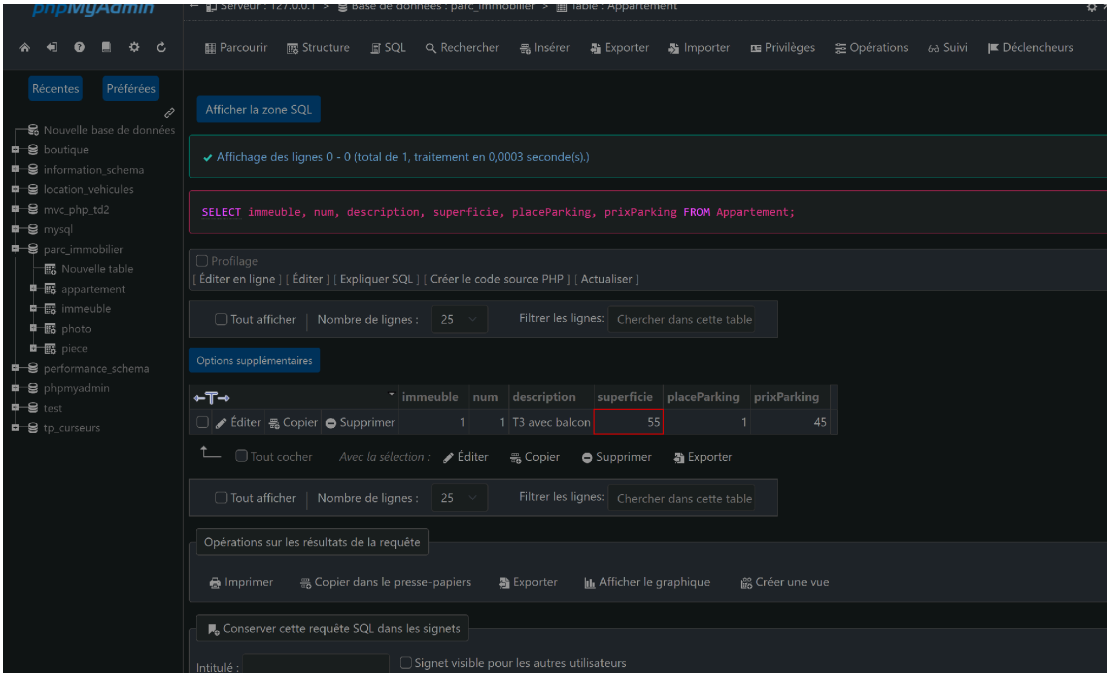
Le trigger refuse un prix de parking alors qu'il n'y a pas de place.

Question 2 : superficie = somme des pièces

La superficie d'un appartement doit égaler la somme des superficies de ses pièces. On crée un trigger AFTER INSERT sur Piece qui ajoute la superficie de la nouvelle pièce à celle de l'appartement. La variable de session @maj_auto autorise cette mise à jour automatique ; elle servira à la question 3.

```
CREATE TRIGGER trg_piece_apres_insert AFTER INSERT ON Piece
FOR EACH ROW
BEGIN
    SET @maj_auto = 1;
    UPDATE Appartement
    SET superficie = superficie + COALESCE(NEW.superficie, 0)
    WHERE immeuble = NEW.immeuble AND num = NEW.appartement;
    SET @maj_auto = 0;
END
```

Test : on insère trois pièces de 25, 18 et 12 m². La superficie de l'appartement passe automatiquement à 55 m².



La superficie de l'appartement est calculée toute seule : 25 + 18 + 12 = 55.

Question 3 : trigger adapté

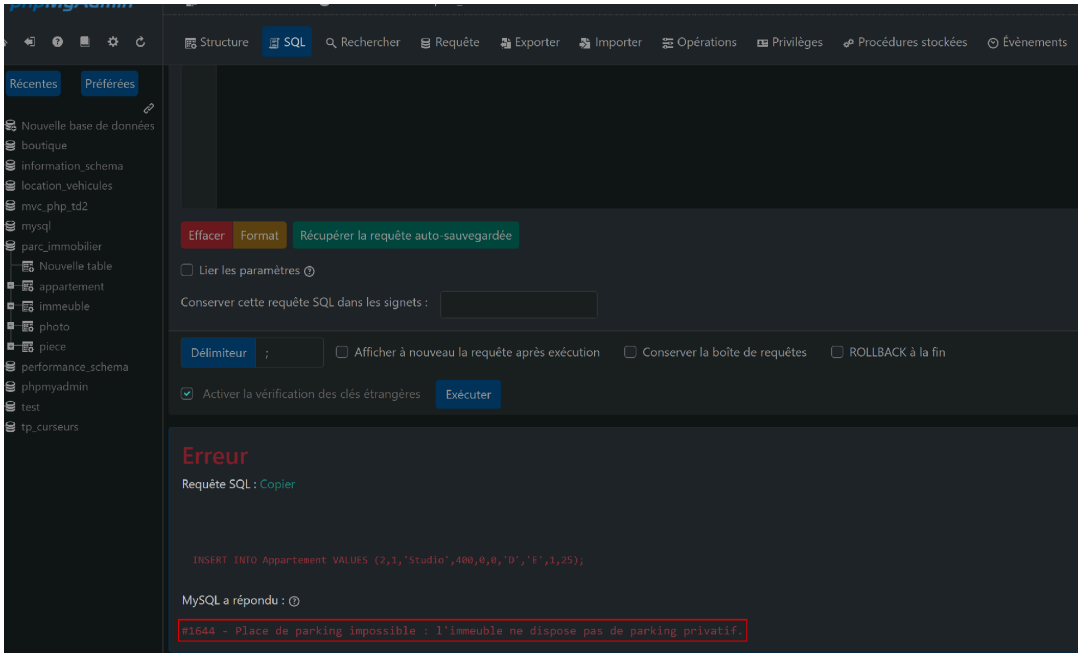
On adapte le trigger de la question 1 pour ajouter trois règles : interdire une place de parking si l'immeuble n'a pas de parking privatif, initialiser la superficie à 0 à la création de l'appartement, et empêcher toute modification manuelle de la superficie (puisqu'elle est désormais calculée automatiquement).

```
CREATE TRIGGER trg_appartement_avant_insert BEFORE INSERT ON Appartement
FOR EACH ROW
BEGIN
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Un appartement sans place de parking ne peut pas avoir de
prix de parking.';
    END IF;
    IF NEW.placeParking = 1 AND NEW.prixParking IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Un appartement avec place de parking doit avoir un prix
de parking.';
    END IF;
    -- place de parking interdite si l'immeuble n'a pas de parking privatif
    IF NEW.placeParking = 1
       AND (SELECT parkingPrivatif FROM Immeuble WHERE id = NEW.immeuble) = 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Place de parking impossible : l''immeuble ne dispose pas
de parking privatif.';
    END IF;
    SET NEW.superficie = 0;   -- superficie initialisee a 0
END

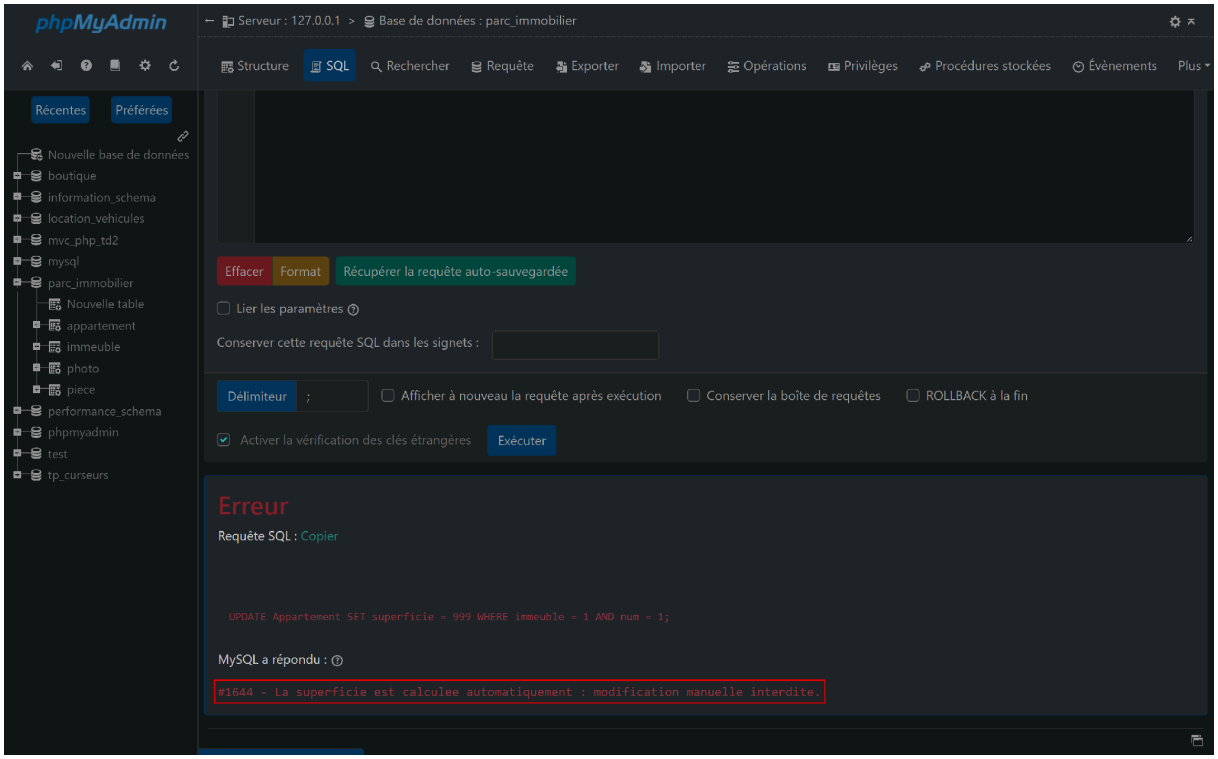
CREATE TRIGGER trg_appartement_avant_update BEFORE UPDATE ON Appartement
FOR EACH ROW
BEGIN
    -- superficie calculee automatiquement : modification manuelle interdite
    IF NEW.superficie <> OLD.superficie AND COALESCE(@maj_auto, 0) = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La superficie est calculee automatiquement : modification
manuelle interdite.';
    END IF;
END
```

Test 1 : on crée une place de parking dans un immeuble qui n'a pas de parking privatif. Refus.



Place de parking refusée car l'immeuble n'a pas de parking privatif.

Test 2 : on essaie de modifier à la main la superficie d'un appartement. Refus également.



La modification manuelle de la superficie est bloquée.

Question 4 : mise à jour et suppression d'une pièce

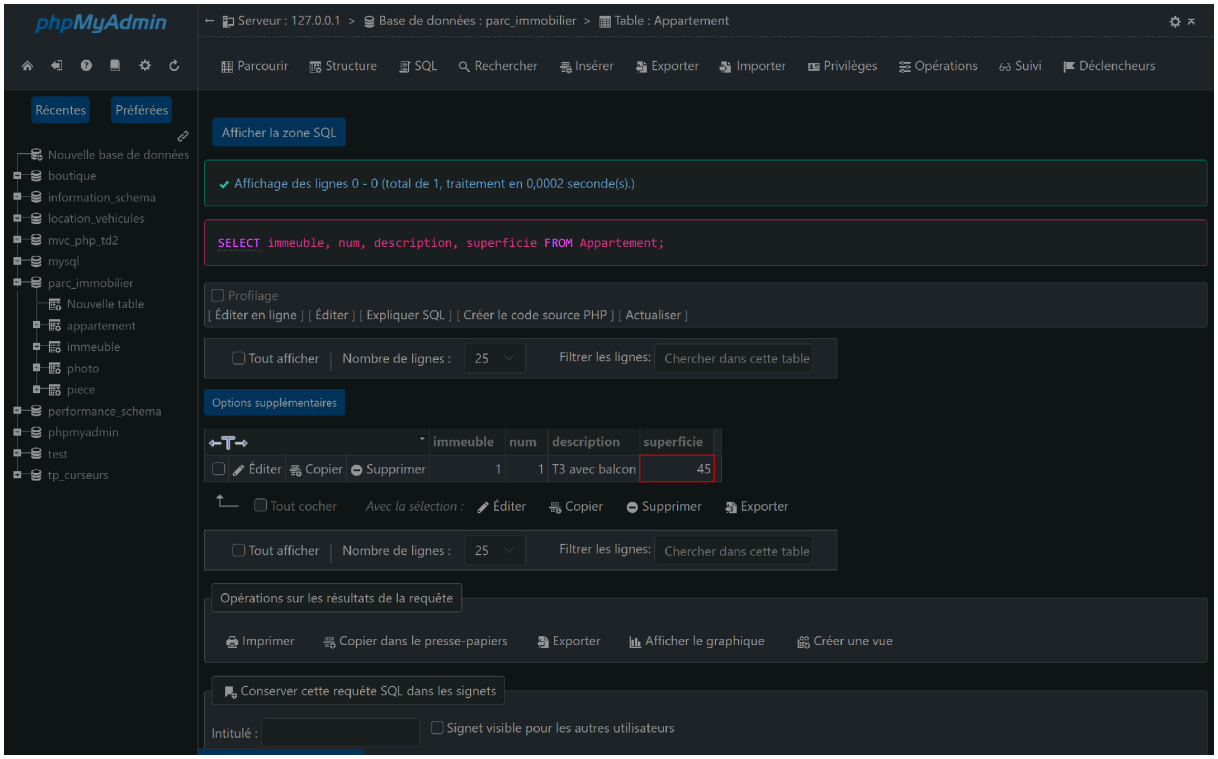
En s'inspirant de la question 2, on met aussi à jour la superficie à la modification et à la suppression d'une pièce.

```
CREATE TRIGGER trg_piece_apres_update AFTER UPDATE ON Piece
FOR EACH ROW
BEGIN
    SET @maj_auto = 1;
    UPDATE Appartement SET superficie = superficie - COALESCE(OLD.superficie, 0)
    WHERE immeuble = OLD.immeuble AND num = OLD.appartement;
    UPDATE Appartement SET superficie = superficie + COALESCE(NEW.superficie, 0)
    WHERE immeuble = NEW.immeuble AND num = NEW.appartement;
    SET @maj_auto = 0;
END

CREATE TRIGGER trg_piece_apres_delete AFTER DELETE ON Piece
FOR EACH ROW
BEGIN
    SET @maj_auto = 1;
    UPDATE Appartement SET superficie = superficie - COALESCE(OLD.superficie, 0)
    WHERE immeuble = OLD.immeuble AND num = OLD.appartement;
```

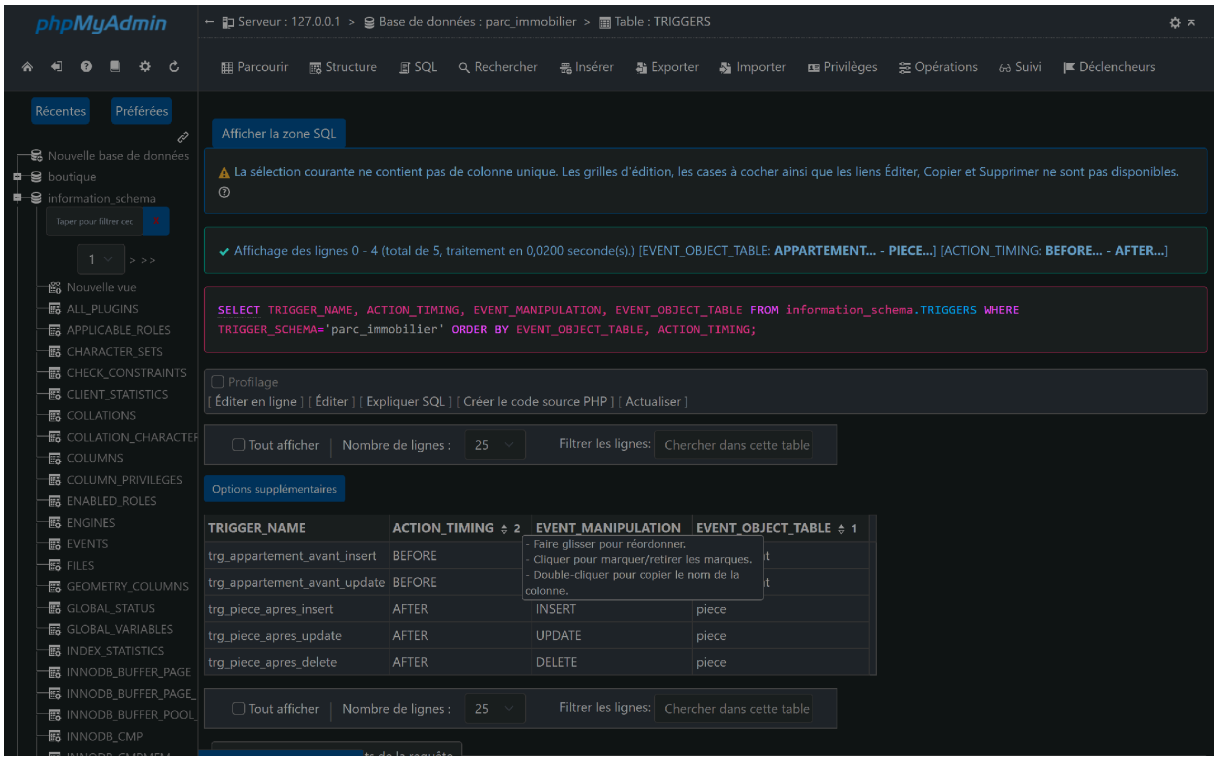
```
    SET @maj_auto = 0;
END
```

Test : on agrandit la cuisine (12 → 20 m²) puis on supprime la chambre (18 m²). La superficie de l'appartement suit : 55 → 63 → 45 m².



Après modification puis suppression d'une pièce, la superficie reste juste (45 m²).

Au final, la base compte cinq triggers sur les tables Appartement et Piece.



Les cinq triggers de la base parc_immobilier.

Exercice 2 : des produits au prix variant au fil du temps

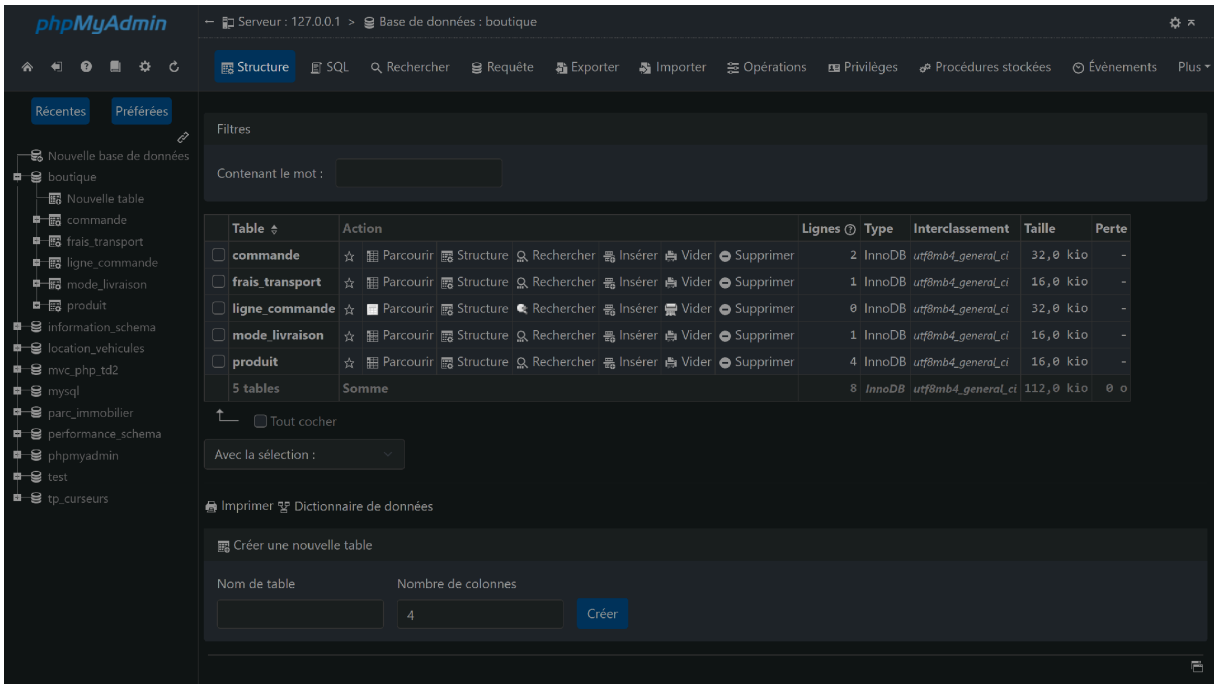
On gère une petite boutique qui vend des produits physiques (des livres) et des formations. Le champ formation de la table Produit vaut 1 pour une formation et 0 pour un livre.

Question 1 : création des tables

On crée les cinq tables du modèle : Produit, Mode_Livraison, Frais_Transport, Commande et Ligne_Commande.

```
CREATE TABLE Produit(
    code INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    libelle VARCHAR(100) NOT NULL, prix DOUBLE NOT NULL,
    formation TINYINT(1) NOT NULL, poids DOUBLE
);
CREATE TABLE Mode_Livraison(
    num INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    libelle VARCHAR(100) NOT NULL
```

```
);
CREATE TABLE Frais_Transport(
    livraison INT(11) NOT NULL, num INT(11) NOT NULL,
    prix DOUBLE NOT NULL, poidsMin DOUBLE NOT NULL, poidsMax DOUBLE NOT NULL,
    CONSTRAINT pk_frais_transport PRIMARY KEY (livraison, num),
    CONSTRAINT fk_frais_livraison FOREIGN KEY (livraison) REFERENCES
Mode_Livraison(num)
);
CREATE TABLE Commande(
    num INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    destinataire VARCHAR(100) NOT NULL, prixTotal DOUBLE NOT NULL DEFAULT 0,
    livraison INT(11), transport INT(11),
    CONSTRAINT fk_commande_transport
        FOREIGN KEY (livraison, transport) REFERENCES Frais_Transport(livraison, num)
);
CREATE TABLE Ligne_Commande(
    commande INT(11) NOT NULL, produit INT(11) NOT NULL,
    prixu DOUBLE NOT NULL, quantite INT(11) NOT NULL, prixt DOUBLE NOT NULL,
    CONSTRAINT pk_ligne_commande PRIMARY KEY (commande, produit),
    CONSTRAINT fk_ligne_commande FOREIGN KEY (commande) REFERENCES Commande(num),
    CONSTRAINT fk_ligne_produit  FOREIGN KEY (produit)  REFERENCES Produit(code)
);
```



Les cinq tables créées dans la base boutique.

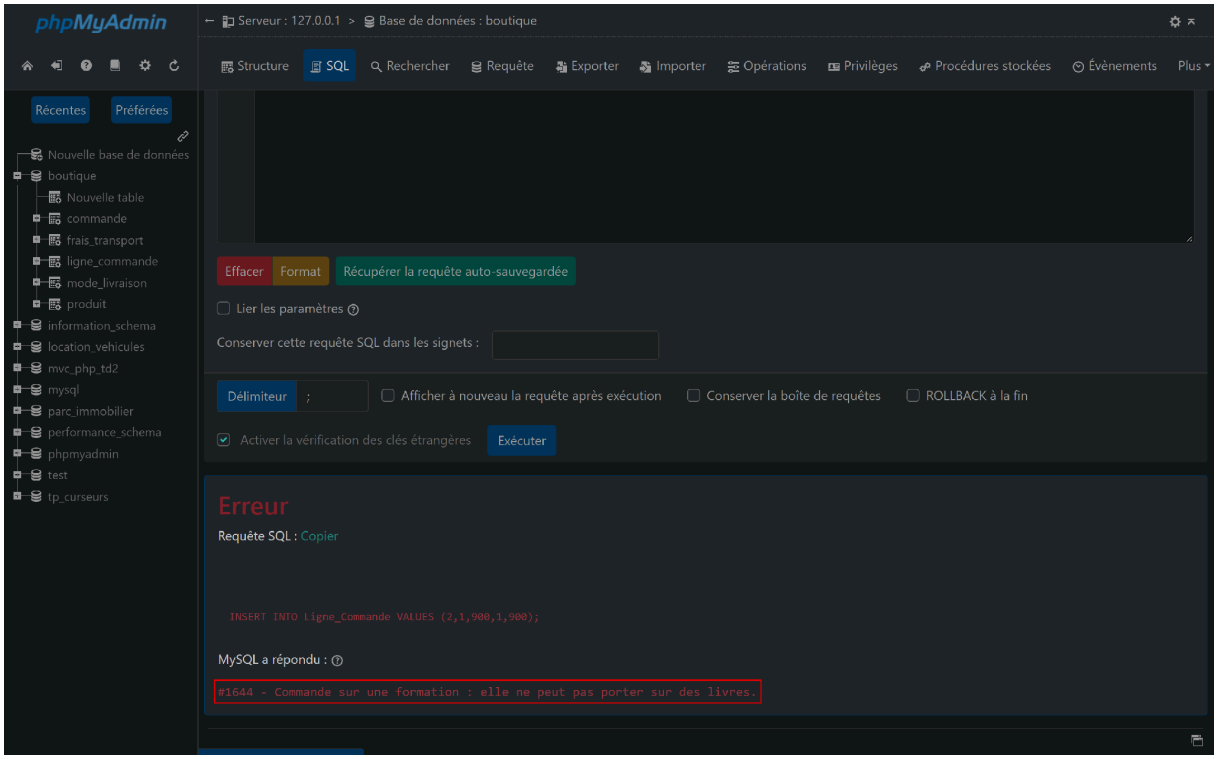
Question 2 : une formation exclut les livres

Une commande qui porte sur une formation ne peut pas porter sur un livre. On écrit un trigger BEFORE INSERT sur Ligne_Commande : selon le type du produit ajouté, il vérifie qu'aucun produit de l'autre type n'est déjà présent dans la commande.

```
CREATE TRIGGER trg_ligne_avant_insert BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    DECLARE estFormation TINYINT;
    SELECT formation INTO estFormation FROM Produit WHERE code = NEW.produit;

    IF estFormation = 1 THEN
        -- Q2 : pas de livre dans une commande qui porte sur une formation
        IF EXISTS (SELECT 1 FROM Ligne_Commande lc JOIN Produit p ON lc.produit =
p.code
                   WHERE lc.commande = NEW.commande AND p.formation = 0) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Commande sur une formation : elle ne peut pas porter
sur des livres.';
        END IF;
    ELSE
        -- Q3 : pas de formation dans une commande qui porte sur des livres
        IF EXISTS (SELECT 1 FROM Ligne_Commande lc JOIN Produit p ON lc.produit =
p.code
                   WHERE lc.commande = NEW.commande AND p.formation = 1) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Commande sur des livres : elle ne peut pas porter sur
une formation.';
        END IF;
    END IF;
END
```

Test : une commande contient déjà un livre ; on tente d'y ajouter une formation. Refus.

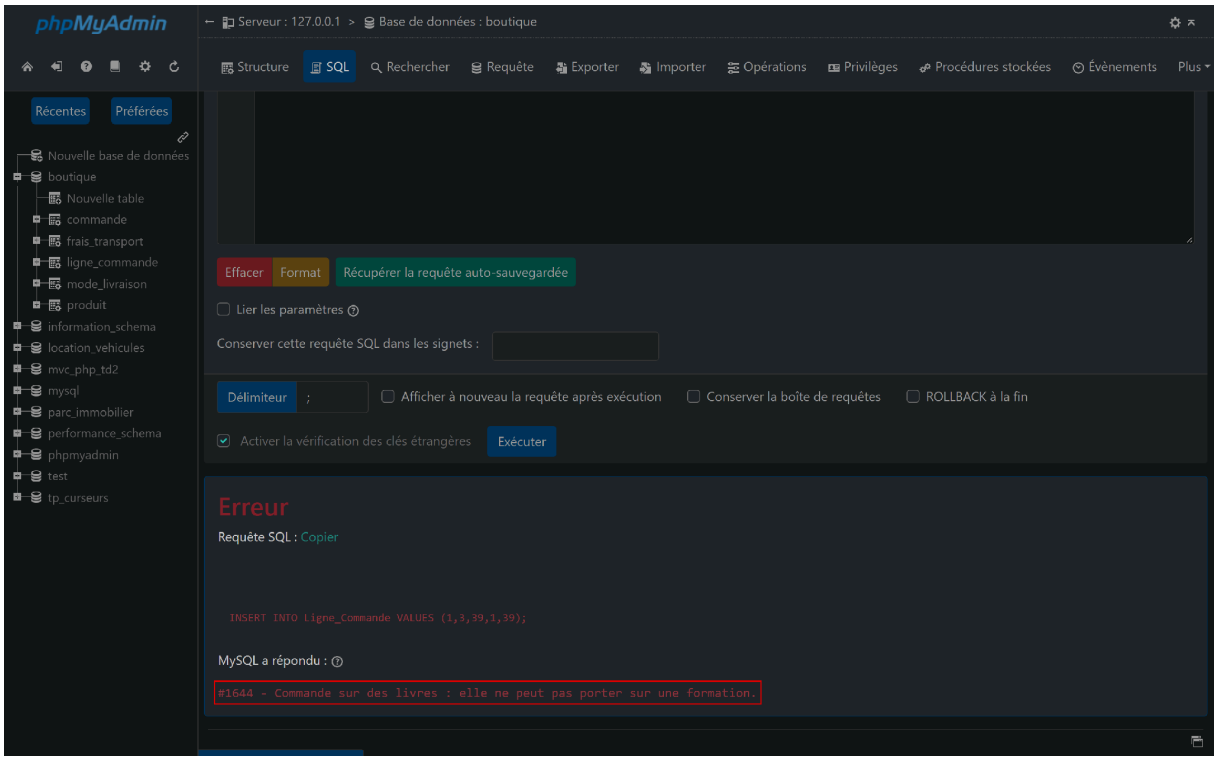


On ne peut pas ajouter une formation à une commande qui contient des livres.

Question 3 : la contrainte réciproque

C'est la règle inverse : une commande qui porte sur des livres ne peut pas porter sur une formation. Elle est déjà gérée par la branche ELSE du même trigger (ci-dessus). On la vérifie en sens inverse.

Test : une commande contient une formation ; on tente d'y ajouter un livre. Refus.



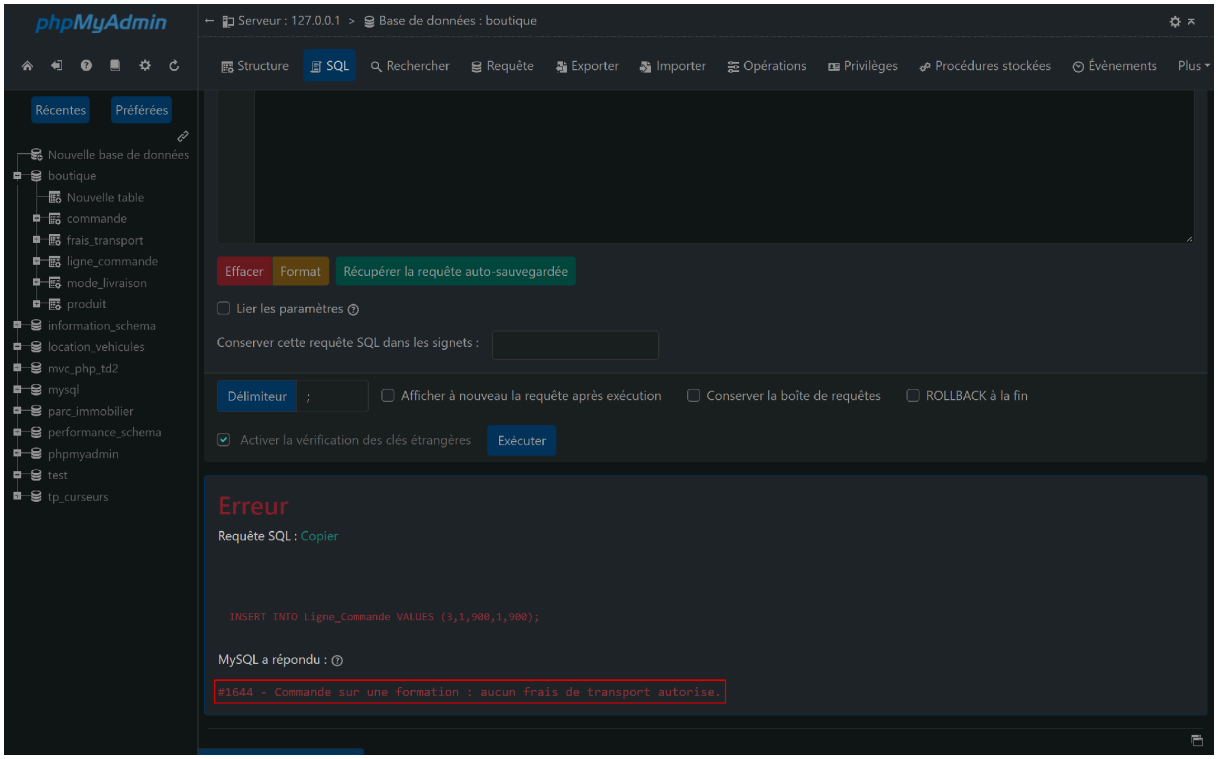
On ne peut pas ajouter un livre à une commande qui contient une formation.

Question 4 : une formation n'a pas de frais de transport

On adapte le même trigger : si la commande porte sur une formation, elle ne doit pas avoir de frais de transport. On ajoute cette vérification dans la branche formation.

```
    -- a inserer dans la branche  IF estFormation = 1 THEN ...
    IF (SELECT transport FROM Commande WHERE num = NEW.commande) IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Commande sur une formation : aucun frais de transport
autorise.';
    END IF;
```

Test : une commande a des frais de transport ; on tente d'y ajouter une formation. Refus.



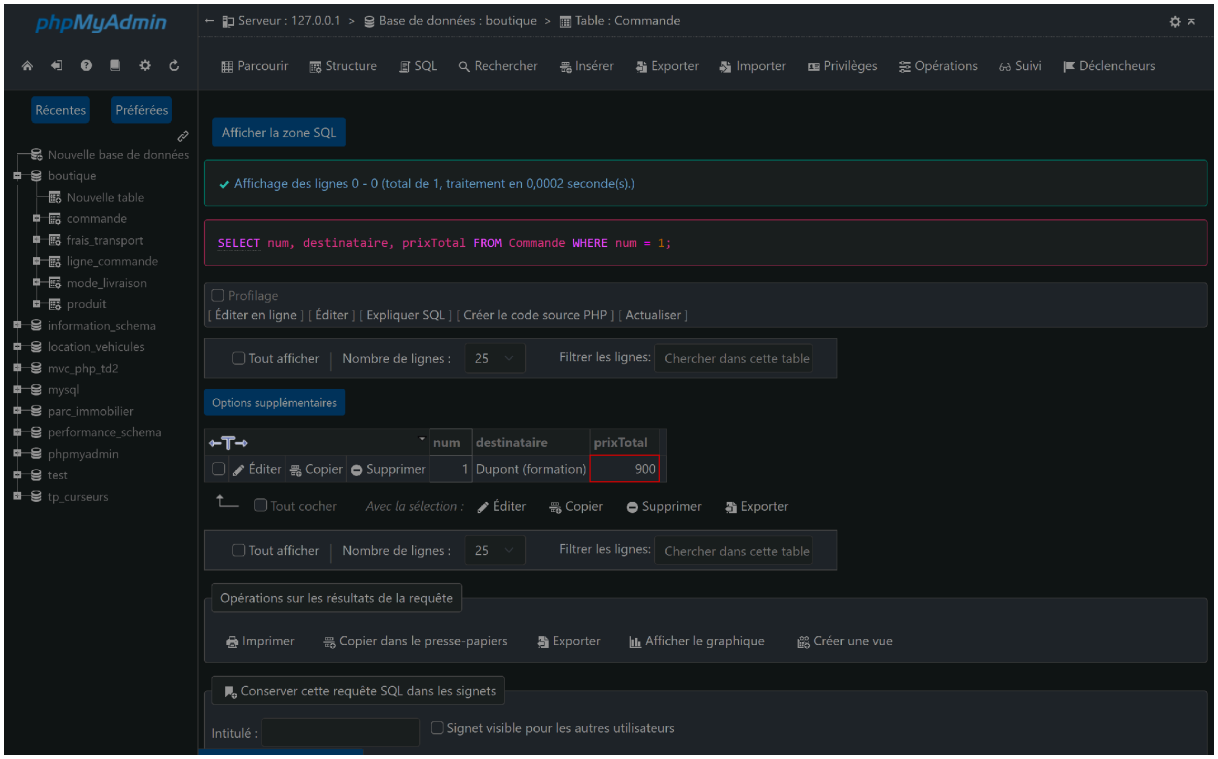
Une formation est refusée dans une commande qui a des frais de transport.

Question 5 : prix total automatique

Le prix total de la commande se met à jour tout seul à chaque ajout de ligne. On crée un trigger AFTER INSERT sur Ligne_Commande qui recalcule la somme des sous-totaux.

```
CREATE TRIGGER trg_ligne_apres_insert AFTER INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    UPDATE Commande
    SET prixTotal = (SELECT COALESCE(SUM(prixt), 0)
                     FROM Ligne_Commande WHERE commande = NEW.commande)
    WHERE num = NEW.commande;
END
```

Test : on ajoute une formation à 900 € à une commande. Son prix total passe directement à 900 €.



Le prix total de la commande est mis à jour automatiquement.

Question 6 : cas non traités

Les questions 1 à 5 ne couvrent pas tout. Restent notamment :

- la modification et la suppression d'une ligne de commande ne recalculent pas le prix total ;

- le changement de type d'un produit (formation ↔ livre) une fois des lignes créées n'est pas contrôlé ;

- l'ajout de frais de transport sur une commande qui contient déjà une formation (le contrôle se fait à l'insertion de la ligne, pas à la mise à jour de la commande) ;

- la cohérence entre le prix unitaire de la ligne et le prix courant du produit ;

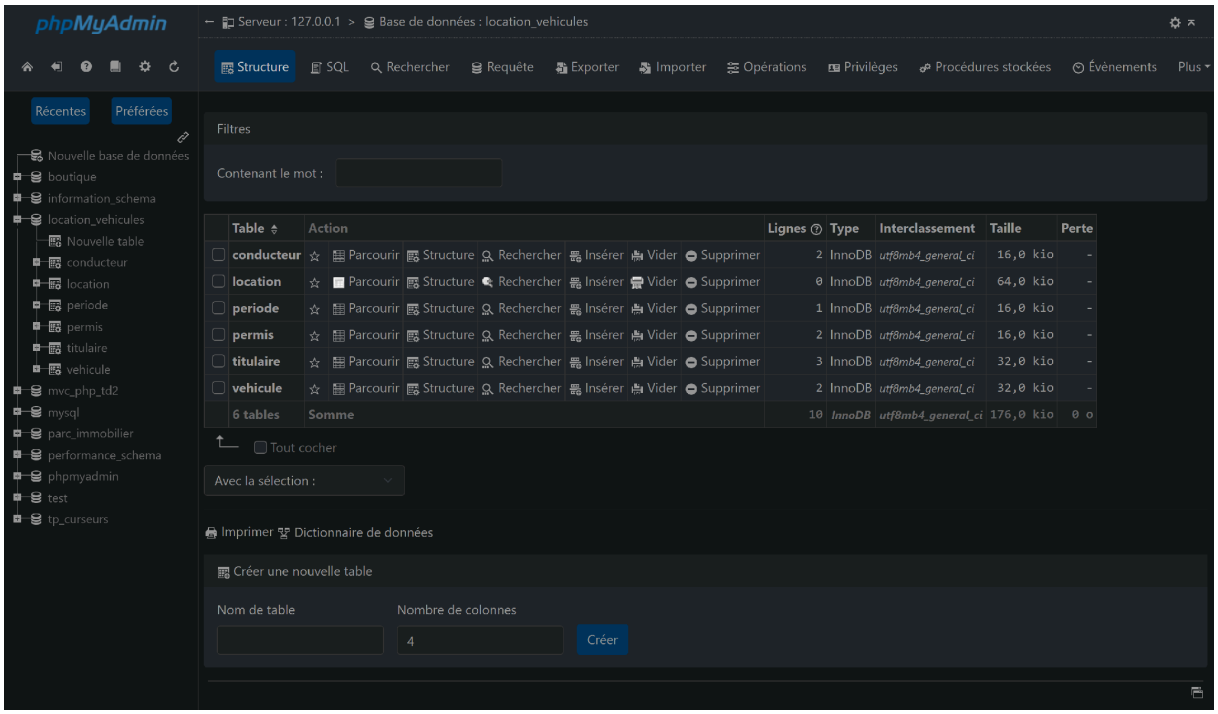
- le respect de la tranche de poids (poidsMin / poidsMax) des frais de transport selon le poids de la commande.

Exercice 3 : location de véhicules

On gère un parc de location. Un Conducteur possède un ou plusieurs permis (table Titulaire), chaque Vehicule exige un permis précis, et une Location associe un conducteur, un véhicule et une période.

Question 1 : création des tables

```
CREATE TABLE Permis(
    id INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY, type VARCHAR(5) NOT NULL
);
CREATE TABLE Conducteur(
    id INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    civilite VARCHAR(5), prenom VARCHAR(50) NOT NULL, nom VARCHAR(50) NOT NULL
);
CREATE TABLE Titulaire(                 -- un conducteur possede plusieurs permis
    conducteur INT(11) NOT NULL, permis INT(11) NOT NULL,
    CONSTRAINT pk_titulaire PRIMARY KEY (conducteur, permis),
    CONSTRAINT fk_titulaire_conducteur FOREIGN KEY (conducteur) REFERENCES
Conducteur(id),
    CONSTRAINT fk_titulaire_permis     FOREIGN KEY (permis)     REFERENCES Permis(id)
);
CREATE TABLE Vehicule(
    id INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    immatriculation VARCHAR(15) NOT NULL,
    permis INT(11) NOT NULL,            -- permis requis pour ce vehicule
    CONSTRAINT fk_vehicule_permis FOREIGN KEY (permis) REFERENCES Permis(id)
);
CREATE TABLE Periode(
    debut DATETIME NOT NULL, fin DATETIME NOT NULL,
    CONSTRAINT pk_periode PRIMARY KEY (debut, fin)
);
CREATE TABLE Location(
    id INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    vehicule INT(11) NOT NULL, conducteur INT(11) NOT NULL,
    debut DATETIME NOT NULL, fin DATETIME NOT NULL,
    CONSTRAINT fk_location_vehicule   FOREIGN KEY (vehicule)   REFERENCES
Vehicule(id),
    CONSTRAINT fk_location_conducteur FOREIGN KEY (conducteur) REFERENCES
Conducteur(id),
    CONSTRAINT fk_location_periode    FOREIGN KEY (debut, fin) REFERENCES
Periode(debut, fin)
);
```



Les six tables de la base location_vehicules.

Question 2 : le conducteur doit avoir le bon permis

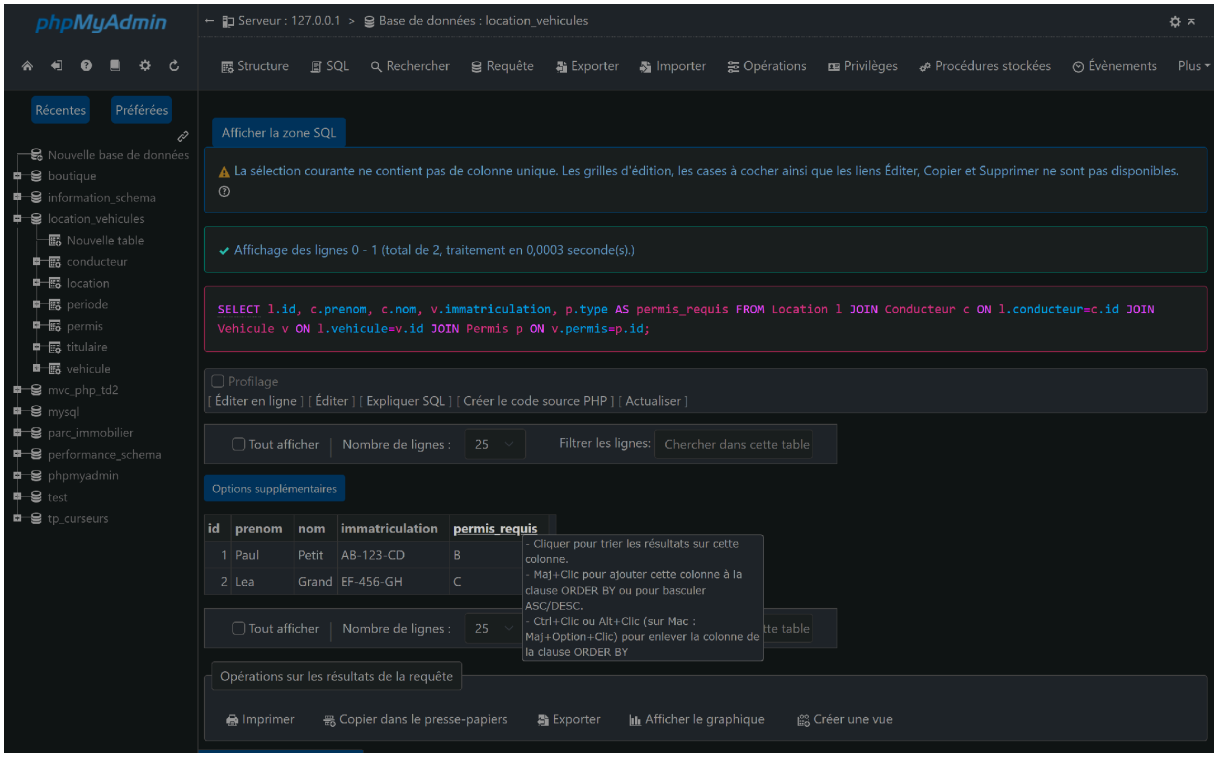
À chaque insertion ou modification d'une location, on vérifie que le conducteur possède bien le permis exigé par le véhicule. Deux triggers (BEFORE INSERT et BEFORE UPDATE) font ce contrôle.

```
CREATE TRIGGER trg_location_avant_insert BEFORE INSERT ON Location
FOR EACH ROW
BEGIN
    DECLARE permisRequis INT;
    SELECT permis INTO permisRequis FROM Vehicule WHERE id = NEW.vehicule;
    IF NOT EXISTS (SELECT 1 FROM Titulaire
                   WHERE conducteur = NEW.conducteur AND permis = permisRequis) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le conducteur ne possede pas le permis requis pour ce
vehicule.';
    END IF;
END

CREATE TRIGGER trg_location_avant_update BEFORE UPDATE ON Location
FOR EACH ROW
BEGIN
    DECLARE permisRequis INT;
    SELECT permis INTO permisRequis FROM Vehicule WHERE id = NEW.vehicule;
    IF NOT EXISTS (SELECT 1 FROM Titulaire
```

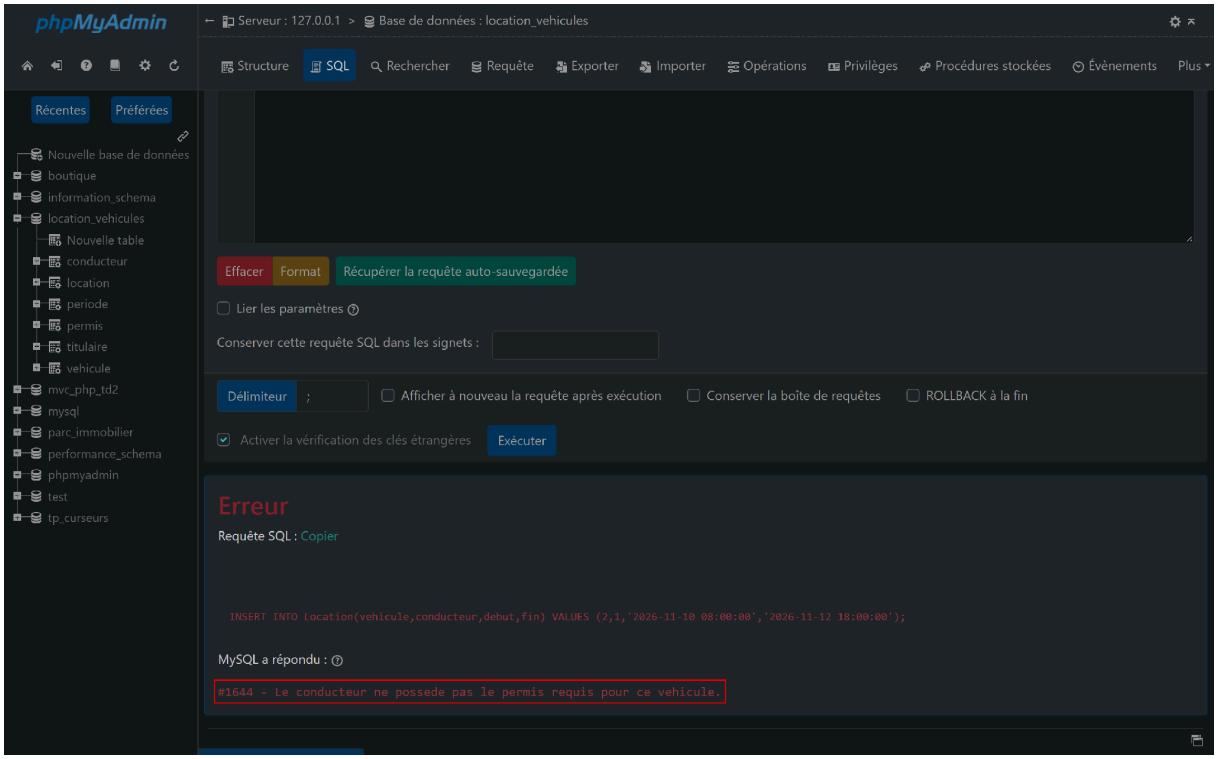
```
                    WHERE conducteur = NEW.conducteur AND permis = permisRequis) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le conducteur ne possede pas le permis requis pour ce
vehicule.';
    END IF;
END
```

Test 1 : Paul (permis B) loue la voiture, et Léa (permis B et C) loue le camion. Les deux locations valides sont acceptées.



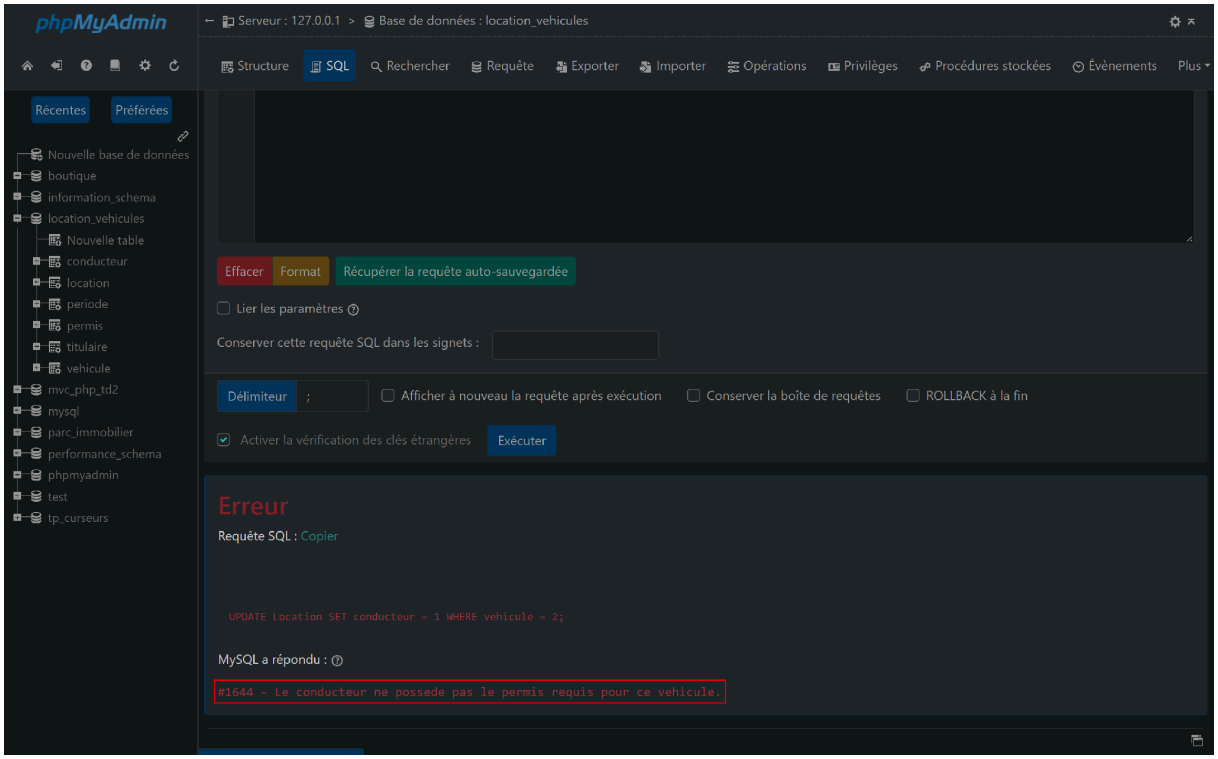
Deux locations valides : chaque conducteur a le permis du véhicule.

Test 2 : Paul tente de louer le camion, qui exige le permis C qu'il n'a pas. Refus à l'insertion.



Paul ne peut pas louer le camion : il n'a pas le permis C.

Test 3 : on essaie de réaffecter la location du camion à Paul par une modification. Le trigger BEFORE UPDATE refuse aussi.



Le contrôle fonctionne aussi à la modification d'une location.

Conclusion

Ce TP montre comment les triggers complètent les contraintes classiques pour faire respecter des règles de gestion : cohérence des données (prix du parking), calculs automatiques (superficie, prix total), et contraintes conditionnelles (formation contre livres, permis requis).

On a utilisé NEW et OLD pour accéder aux lignes, SIGNAL SQLSTATE '45000' pour bloquer les opérations interdites, et une variable de session pour distinguer les mises à jour automatiques des modifications manuelles. Tous les triggers ont été testés sous phpMyAdmin.